

Warming-up 프로그램 1

2026-2 컴퓨터 그래픽스

(각 문제는 3점~5점으로 채점됩니다.)

1. 행렬 다루기

- 4 X 4 행렬의 연산하는 프로그램을 구현한다.
- 연산을 할 두 개의 행렬에는 한 자리 숫자 (1에서 9 사이의 숫자)를 랜덤하게 설정한다. (사용자 입력x, 자동 배정)
- 다음의 명령어로 수행한다. 종료 명령어를 입력할 때까지 명령을 연속적으로 수행할 수 있도록 한다.
 - m: 행렬의 곱셈
 - a: 행렬의 덧셈
 - d: 행렬의 뺄셈
 - r: 행렬식의 값 (Determinant) → 입력한 2개의 행렬의 행렬식 값을 모두 출력한다.
 - t: 전치 행렬(Transposed matrix)과 그 행렬식의 값, 설정된 두 개의 행렬에 모두 적용한다.
 - e: 각 행에서 최소값을 찾아 그 값을 해당 행의 값에서 뺀다. → 다시 누르면 원래대로 출력한다.
 - f: 각 열에서 최대값을 찾아 그 값을 해당 열의 값에 더한다. → 다시 누르면 원래대로 출력한다.
 - +/-: 행렬의 모든 값에 +1/-1을 진행하고 값의 범위는 10으로 모듈라 연산하여 0~9 범위로 한다. (즉, 값은 $8 \leftrightarrow 9 \leftrightarrow 0 \leftrightarrow 1 \dots$)
 - s: 행렬의 값을 새로 랜덤하게 설정한다.
 - q: 프로그램 종료

- 점수: 총 4점 (각 0.5점, +/-는 한 묶음으로 채점)
- s와 q는 기본 명령어
- 기본 명령어: 배점은 없지만 실행 안되면 전체 점수를 받지 못함.

실행 예)

(명령어) m:
$$\begin{pmatrix} 2 & 1 & 5 & 3 \\ 7 & 4 & 6 & 4 \\ 5 & 2 & 2 & 1 \\ 8 & 7 & 2 & 3 \end{pmatrix} \cdot \begin{pmatrix} 4 & 7 & 2 & 1 \\ 7 & 8 & 2 & 9 \\ 6 & 5 & 5 & 1 \\ 8 & 4 & 2 & 2 \end{pmatrix} = \begin{pmatrix} 69 & 59 & 37 & 22 \\ 124 & 127 & 60 & 57 \\ 54 & 65 & 26 & 27 \\ 117 & 134 & 46 & 79 \end{pmatrix}$$

(명령어) a:
$$\begin{pmatrix} 2 & 1 & 5 & 3 \\ 7 & 4 & 6 & 4 \\ 5 & 2 & 2 & 1 \\ 8 & 7 & 2 & 3 \end{pmatrix} + \begin{pmatrix} 4 & 7 & 2 & 1 \\ 7 & 8 & 2 & 9 \\ 6 & 5 & 5 & 1 \\ 8 & 4 & 2 & 2 \end{pmatrix} = \begin{pmatrix} 6 & 8 & 7 & 4 \\ 14 & 12 & 8 & 13 \\ 11 & 7 & 7 & 2 \\ 16 & 11 & 4 & 5 \end{pmatrix}$$

(명령어 입력) r:
$$\begin{vmatrix} 2 & 1 & 5 & 3 \\ 7 & 4 & 6 & 4 \\ 5 & 2 & 2 & 1 \\ 8 & 7 & 2 & 3 \end{vmatrix} = 3$$

(명령어) t:
$$\begin{pmatrix} 2 & 1 & 5 & 3 \\ 7 & 4 & 6 & 4 \\ 5 & 2 & 2 & 1 \\ 8 & 7 & 2 & 3 \end{pmatrix}^T = \begin{pmatrix} 2 & 7 & 5 & 8 \\ 1 & 4 & 2 & 7 \\ 5 & 6 & 2 & 2 \\ 3 & 4 & 1 & 3 \end{pmatrix}$$

(명령어) d:
$$\begin{pmatrix} 2 & 1 & 5 & 3 \\ 7 & 4 & 6 & 4 \\ 5 & 2 & 2 & 1 \\ 8 & 7 & 2 & 3 \end{pmatrix} - \begin{pmatrix} 4 & 7 & 2 & 1 \\ 7 & 8 & 2 & 9 \\ 6 & 5 & 5 & 1 \\ 8 & 4 & 2 & 2 \end{pmatrix} = \begin{pmatrix} -2 & -6 & 3 & 2 \\ 0 & -4 & 4 & -5 \\ -1 & -3 & -3 & 0 \\ 0 & 3 & 0 & 1 \end{pmatrix}$$

2. 파일에서 문자열 읽기

- 문자와 숫자로 구성된 문장을 10줄 저장한 파일을 만든다. (파일 이름 예: data.txt)
- 데이터 파일 이름을 입력 받고, 파일에서 문자열을 읽고 저장한다.
- 읽은 문자열을 출력하고, 공백을 기준으로 단어를 구분한다.
 - 1개 이상의 공백이 있는 경우는 공백을 1개로 취급한다.
 - 알파벳이나 숫자 외의 특수문자들은 (예, / , - * 같은 문자) 구분하지 않고 연결된 값으로 카운트 한다.
- 다음의 명령어를 수행한다.
 - a: 모든 문장의 문자들을 대소문자를 바꿔 출력한다. → 다시 누르면 원래대로 출력한다.
 - b: 각 줄의 단어의 개수를 출력한다. 문장들을 모두 쓰고 각 문장의 맨 뒤에 해당 문장의 단어의 개수를 출력한다.
 - c: 대문자로 시작되는 단어를 찾아 그 단어를 다른 색으로 출력하고, 몇 개 있는지를 계산하여 출력한다. → 다시 누르면 원래대로 출력한다.
 - d: 각 문장 별로 거꾸로 출력하기. → 다시 누르면 원래대로 출력한다.
 - e: 모든 공백에 "*" 문자 삽입하기. → 다시 누르면 원래대로 출력한다.
 - f: 공백을 기준으로 (*가 삽입되어 있다면 *를 공백으로 취급) 모든 단어들을 거꾸로 출력하기. → 다시 누르면 원래대로 출력한다.
 - g: 문자 내부의 특정 문자를 다른 문자로 바꾸기 (바꿀 문자와 새롭게 입력할 문자 입력 받음) → 다시 누르면 원래대로 출력한다.
 - h: 문장에 있는 숫자를 찾아 숫자 뒤에 오는 문장을 다음 줄로 넘긴다. → 다시 누르면 원래대로 출력한다.
 - i: 명령어와 단어를 입력하면, 문장들을 모두 출력하면서 입력 받은 단어를 찾아 다른 색으로 출력하고, 몇 개 있는지를 계산하여 출력한다. (대소문자 구분하지 않는다)
 - j: 문장의 순서를 바꿔서 출력한다. 즉, 1번 문장 → 2번 문장, 2번 문장 → 3번 문장, ... , 9번 문장 → 1번 문장
 - q: 프로그램 종료

점수: 총 5점 (각 0.5점)
q는 기본 명령어

2. 파일에서 문자열 읽기

- 콘솔에서 색 출력하기

```
#include <windows.h>
```

```
SetConsoleTextAttribute (GetStdHandle(STD_OUTPUT_HANDLE), attribute_color); //--attribute_color; WORD 타입으로 색상을 나타내는 숫자
```

- 콘솔에서 제공해주는 색상



- 사용 예)

```
#include <stdio.h>
#include <stdlib.h>
#include <windows.h>
int main()
{
    int i;
    for (i = 0; i < 16; i++)
    {
        unsigned short text = i;
        SetConsoleTextAttribute (GetStdHandle(STD_OUTPUT_HANDLE), text);
        printf("setColor (%d)\n", text);
    }
    return 0;
}
```

```
setColor (1)
setColor (2)
setColor (3)
setColor (4)
setColor (5)
setColor (6)
setColor (7)
setColor (8)
setColor (9)
setColor (10)
setColor (11)
setColor (12)
setColor (13)
setColor (14)
setColor (15)
```

<결과 화면>

2. 파일에서 문자열 읽기

- 예)

input data file name: ***data.txt***

This is Computer Graphics Warming-up program.

1 One 2 Two 3 Three 4 Four

File input output sample program

2026 Fall semester 3D computer graphics

Tech University in SOUTH Korea

AI Convergence College Game and Game engineering department

It was so hot this summer and I'm glad I was able to endure this extreme heat, well.

Computer graphics class this semester is on Mondays, Tuesdays, and Wednesdays.

fall Game Korea DEPARTMENT multi 3D Korea

MonDay TuesDay WednesDay ThursDay FriDay SaturDay SunDay

2. 파일에서 문자열 읽기

input the command: h (예) 2026 Fall semester 3D computer graphics 문장 변경)

2

0

2

6

Fall semester 3

D computer graphics

3. 저장 리스트 만들기 (구조체 데이터 사용)

- 점 (x, y, z) 데이터 값을 저장하는 리스트를 만든다. (점 데이터는 각각 정수이고, 구조체를 사용하도록 한다.)
- 최대 10개의 점 데이터를 저장하도록 한다. (0번: 맨 아래, 9번: 맨 위)
- 리스트에 데이터를 입력하거나 삭제하고 출력하는 명령어를 실행한다.
 - 각 명령어를 입력 받으면 결과 리스트를 다음페이지의 그림과 같이 **항상 10개의 항목을 가진 리스트로 인덱스 번호와 데이터 값을 출력한다.**
- 구현 함수 프로토타입과 명령어:

점수: 총 5점 (각 0.5점, g는 1점)
q는 기본 명령어

 - + x y z: 리스트의 맨 위에 입력 (x, y, z: 숫자)
 - -: 리스트의 맨 위에서 삭제한다.
 - e x y z: 리스트의 맨 아래에 입력 (명령어 +와 반대의 위치, 리스트에 저장된 데이터값이 위로 올라간다.)
 - d: 리스트의 맨 아래에서 삭제한다. (리스트에서 삭제된 칸이 비어있다.)
 - a: 리스트에 저장된 점의 개수를 출력한다.
 - b: 점들의 리스트 위치를 한 칸씩 내려 보낸다. (즉, $0 \rightarrow 9, 1 \rightarrow 0, 2 \rightarrow 1, \dots 9 \rightarrow 8$)
 - c: 리스트를 비운다. 리스트를 비운 후 다시 입력하면 0번부터 저장된다.
 - f: 각 점에서 원점과의 거리를 계산 후, 그 값을 정렬하여 오름차순으로 정렬하여 출력한다. 인덱스 0번부터 빈 칸없이 저장하여 출력한다. 리스트 각 칸의 옆에는 해당 칸의 점과 원점과의 거리를 출력한다. 다시 누르면 원래대로 출력한다.
 - g: 리스트에 저장된 점들에서 두 점간의 모든 조합에 대한 거리를 계산하고 가장 먼 두 점, 가장 가까운 두 점을 출력한다. 이때, 두 점의 좌표 값과 그 점 사이의 거리, 가장 먼 두 점, 가장 가까운 두 점을 출력하고 그 점간의 거리도 출력한다.
 - q: 프로그램을 종료한다.

** 리스트에서 맨 위(인덱스 9번)까지 차고 아래칸(인덱스 0번)이 비어 있으면 다음 데이터 입력할 때는 0번에 입력된다.

** 즉, 10개의 칸을 다 채울 수 있어야 함.

3. 저장 리스트 만들기 (구조체 데이터 사용)

실행 예)

+ 0 1 0

+ 0 1 1

-

e 1 1 1

e 1 0 1

+ 1 1 0

d

→ 1번 그림

→ 2번 그림

→ 3번 그림

→ 4번 그림

→ 5번 그림

→ 6번 그림

→ 7번 그림

데이터 출력 시 아래의 그림과 같이
인덱스 번호와 좌표값을 같이
출력하도록 한다. 순서는 어느쪽
방향이던 무관하다.

1	<table><tr><td>9</td><td></td></tr><tr><td>8</td><td></td></tr><tr><td>7</td><td></td></tr><tr><td>6</td><td></td></tr><tr><td>5</td><td></td></tr><tr><td>4</td><td></td></tr><tr><td>3</td><td></td></tr><tr><td>2</td><td></td></tr><tr><td>1</td><td></td></tr><tr><td>0</td><td>0 1 0</td></tr></table>	9		8		7		6		5		4		3		2		1		0	0 1 0
9																					
8																					
7																					
6																					
5																					
4																					
3																					
2																					
1																					
0	0 1 0																				
2	<table><tr><td>9</td><td></td></tr><tr><td>8</td><td></td></tr><tr><td>7</td><td></td></tr><tr><td>6</td><td></td></tr><tr><td>5</td><td></td></tr><tr><td>4</td><td></td></tr><tr><td>3</td><td></td></tr><tr><td>2</td><td></td></tr><tr><td>1</td><td>0 1 1</td></tr><tr><td>0</td><td>0 1 0</td></tr></table>	9		8		7		6		5		4		3		2		1	0 1 1	0	0 1 0
9																					
8																					
7																					
6																					
5																					
4																					
3																					
2																					
1	0 1 1																				
0	0 1 0																				
3	<table><tr><td>9</td><td></td></tr><tr><td>8</td><td></td></tr><tr><td>7</td><td></td></tr><tr><td>6</td><td></td></tr><tr><td>5</td><td></td></tr><tr><td>4</td><td></td></tr><tr><td>3</td><td></td></tr><tr><td>2</td><td></td></tr><tr><td>1</td><td></td></tr><tr><td>0</td><td>0 1 0</td></tr></table>	9		8		7		6		5		4		3		2		1		0	0 1 0
9																					
8																					
7																					
6																					
5																					
4																					
3																					
2																					
1																					
0	0 1 0																				
4	<table><tr><td>9</td><td></td></tr><tr><td>8</td><td></td></tr><tr><td>7</td><td></td></tr><tr><td>6</td><td></td></tr><tr><td>5</td><td></td></tr><tr><td>4</td><td></td></tr><tr><td>3</td><td></td></tr><tr><td>2</td><td></td></tr><tr><td>1</td><td>0 1 0</td></tr><tr><td>0</td><td>1 1 1</td></tr></table>	9		8		7		6		5		4		3		2		1	0 1 0	0	1 1 1
9																					
8																					
7																					
6																					
5																					
4																					
3																					
2																					
1	0 1 0																				
0	1 1 1																				
5	<table><tr><td>9</td><td></td></tr><tr><td>8</td><td></td></tr><tr><td>7</td><td></td></tr><tr><td>6</td><td></td></tr><tr><td>5</td><td></td></tr><tr><td>4</td><td></td></tr><tr><td>3</td><td></td></tr><tr><td>2</td><td>0 1 0</td></tr><tr><td>1</td><td>1 1 1</td></tr><tr><td>0</td><td>1 0 1</td></tr></table>	9		8		7		6		5		4		3		2	0 1 0	1	1 1 1	0	1 0 1
9																					
8																					
7																					
6																					
5																					
4																					
3																					
2	0 1 0																				
1	1 1 1																				
0	1 0 1																				
6	<table><tr><td>9</td><td></td></tr><tr><td>8</td><td></td></tr><tr><td>7</td><td></td></tr><tr><td>6</td><td></td></tr><tr><td>5</td><td></td></tr><tr><td>4</td><td></td></tr><tr><td>3</td><td>1 1 0</td></tr><tr><td>2</td><td>0 1 0</td></tr><tr><td>1</td><td>1 1 1</td></tr><tr><td>0</td><td>1 0 1</td></tr></table>	9		8		7		6		5		4		3	1 1 0	2	0 1 0	1	1 1 1	0	1 0 1
9																					
8																					
7																					
6																					
5																					
4																					
3	1 1 0																				
2	0 1 0																				
1	1 1 1																				
0	1 0 1																				
7	<table><tr><td>9</td><td></td></tr><tr><td>8</td><td></td></tr><tr><td>7</td><td></td></tr><tr><td>6</td><td></td></tr><tr><td>5</td><td></td></tr><tr><td>4</td><td></td></tr><tr><td>3</td><td>1 1 0</td></tr><tr><td>2</td><td>0 1 0</td></tr><tr><td>1</td><td>1 1 1</td></tr><tr><td>0</td><td></td></tr></table>	9		8		7		6		5		4		3	1 1 0	2	0 1 0	1	1 1 1	0	
9																					
8																					
7																					
6																					
5																					
4																					
3	1 1 0																				
2	0 1 0																				
1	1 1 1																				
0																					

4. 카드 짝 맞추기 게임 만들기

- 격자모양의 보드를 만드는데, 가로와 세로의 크기를 입력 받는다. (단, 범위는 3~6)
 - 가로와 세로의 크기는 달라도 무관하다.
 - 가로와 세로의 크기가 결정되면 그 크기에 맞춰 격자 보드를 그리고 격자칸을 *로 표시한다.
 - 격자칸에는 격자갯수의 절반 개수만큼의 소문자가 각각 2개씩 보드에 무작위로 배치된다.
 - 이때, 격자칸의 개수가 홀수이면, 나머지 한 개는 조커로 배치한다.
 - 이때 격자칸의 개수가 짝수이면, 조커가 없다.
 - 예를들어, 가로x세로가 3x3 → 9개의 격자칸 → 4개의 소문자가 2개씩 배치된다 → 즉, 8개에 배치되면 1개의 격자칸에는 조커가 배치된다.
 - 예를들어, 가로x세로가 4x5 → 20개의 격자칸 → 10개의 소문자가 2개씩 배치된다 → 즉, 20개에 배치되고 조커는 없다.
- 해당 갯수의 다른 소문자를 각각 2개씩 보드에 무작위로 배치한다. 나머지 한 개는 조커로 어느 카드와도 매치된다.
 - 보드 위쪽에 a b c d e, 좌측에 1 2 3 4 5를 표시하여 칸의 위치를 알 수 있게 한다.
 - 2개의 격자를 선택한다. 엔터키를 치면 선택된 칸의 문자가 보여진다.
 - 두 문자가 같으면 그 칸에는 문자가 대문자로 변경되어 계속 그려지고, 문자가 다르면 다시 가려진다. 문자의 색상 변경한다.
 - 특정 횟수만큼 진행할 수 있게 하고, 점수를 배점하여 출력한다. (횟수, 점수 배점은 각자 결정하기)
 - 선택한 2개중에 조커가 있으면 나머지 한 개의 카드가 열린다.
 - 명령어:
 - r: 게임을 리셋하고 다시 시작한다.
 - h: 보드칸의 모든 문자들을 잠시 보여주고 다시 원래대로 출력한다. (힌트 키)
 - q: 프로그램 종료
- 보드칸 선택 방법
 - 행렬의 번호를 입력 받아 선택하기

점수: 4점

- 게임 기본/명령어: 2점
- 보드 맞추기: 2점

4. 카드 짝 맞추기 게임 만들기

- 실행 예) 가로x세로 입력: 3 4

	a	b	c	d
1	a	*	*	*
2	*	*	*	*
3	*	*	c	*

	a	b	c	d
1	*	*	*	*
2	*	*	*	*
3	*	*	*	*

	a	b	c	d
1	a	*	*	*
2	*	*	*	a
3	*	*	*	*

	a	b	c	d
1	A	*	*	*
2	*	*	*	A
3	*	*	*	*

칸수가 짝수라서 (12칸) 조커가 없다.

- 실행 예) 가로x세로 입력: 5 5

	a	b	c	d	e
1	a	*	*	*	*
2	*	*	*	*	*
3	*	*	c	*	*
4	*	*	*	*	*
5	*	*	*	*	*

	a	b	c	d	e
1	*	*	*	*	*
2	*	*	*	*	*
3	*	*	*	*	*
4	*	*	*	*	*
5	*	*	*	*	*

	a	b	c	d	e
1	a	*	*	*	*
2	*	*	*	a	*
3	*	*	*	*	*
4	*	*	*	*	*
5	*	*	*	*	*

	a	b	c	d	e
1	A	*	*	*	*
2	*	*	*	A	*
3	*	*	*	*	*
4	*	*	*	*	*
5	*	*	*	*	*

	a	b	c	d	e
1	A	*	*	@	*
2	*	*	*	A	*
3	*	*	*	*	*
4	*	*	*	*	*
5	*	d	*	*	*

	a	b	c	d	e
1	A	*	*	@	*
2	*	*	*	A	*
3	*	*	*	*	*
4	*	*	*	*	D
5	*	D	*	*	*

입력: a1 c3 → 불일치 입력: a1 d2 → 일치 (대문자로 변경) 입력: b5 d2 → 조커 → 자동으로 카드 열림